



## UNIVERSIDADE PAULISTA - UNIP

Ciência da Computação - 8º semestre  
Desenvolvimento de Sistemas Distribuídos

|                         |         |
|-------------------------|---------|
| Aluísio Pereira Alves   | N135891 |
| Enzo Orlandi Gomes      | G788EJ5 |
| Erick de Brito Carvalho | G78HED3 |
| Kayky Crespo dos Santos | G839226 |

# PEDIDO JÁ

## Relatório Técnico do Sistema

API de Pedidos, persistência PostgreSQL, interface React e infraestrutura  
Docker

Entrega 1

## Resumo executivo

O Pedido Já é uma aplicação distribuída de primeira entrega cujo domínio foi mantido propositalmente pequeno para concentrar a avaliação na arquitetura. A solução permite criar, consultar, listar e alterar o status de pedidos, persiste os dados em PostgreSQL e oferece uma interface web React integrada à API FastAPI.

A implantação final utiliza exatamente dois serviços Docker: pedidos e postgres. A aplicação contém as camadas lógicas API/Router, Service e Repository no mesmo processo, enquanto o PostgreSQL executa em container separado, com volume persistente. O cliente acessa somente a porta 8000 publicada pelo serviço pedidos.

**Resultado central:** Um único comando - `docker compose up -d --build` - constrói a SPA, prepara o backend, sobe o PostgreSQL, aplica as migrations Alembic e disponibiliza o sistema completo em `http://localhost:8000`.

| Item                | Implementação                                                              |
|---------------------|----------------------------------------------------------------------------|
| API                 | FastAPI + Uvicorn, HTTP/JSON, OpenAPI/Swagger                              |
| Regras de negócio   | Service dedicado; valor_total calculado com Decimal; status inicial CRIADO |
| Persistência        | PostgreSQL 16 + SQLAlchemy 2.x + Alembic                                   |
| Frontend            | React + TypeScript + Vite, servido pela própria FastAPI                    |
| Containers          | pedidos + postgres                                                         |
| Persistência Docker | volume nomeado postgres_data                                               |
| Execução            | <code>docker compose up -d --build</code>                                  |

# Sumário

Navegação estruturada do relatório técnico.

## 1. Contexto, objetivo e escopo

## 2. Atendimento aos requisitos da Entrega 1

## 3. Arquitetura do sistema

### 3.1 Arquitetura física

### 3.2 Arquitetura lógica interna

### 3.3 Fluxo de criação de um pedido

## 4. Organização do repositório

## 5. Backend FastAPI

### 5.1 Inicialização da aplicação

### 5.2 Banco e sessão por requisição

## 6. Modelo de domínio e regras de negócio

### 6.1 Regras essenciais

## 7. API REST e contratos HTTP

### 7.1 GET /health

### 7.2 POST /pedidos

### 7.3 GET /pedidos/{id}

### 7.4 GET /pedidos

### 7.5 PATCH /pedidos/{id}/status

### 7.6 Documentação OpenAPI

## 8. Persistência PostgreSQL e Alembic

### 8.1 Tabelas

### 8.2 Estrutura lógica da tabela pedidos

### 8.3 Migrations

### 8.4 Persistência após reinício da aplicação

## 9. Frontend React

### 9.1 Rotas públicas

### 9.2 Rotas administrativas e técnicas

### 9.3 Integração com a API

### 9.4 Evidências visuais da interface

## 10. Docker e processo de inicialização

### 10.1 Dockerfile multi-stage

### 10.2 Docker Compose

### 10.3 Inicialização automatizada

## 11. Configuração por variáveis de ambiente

## 12. Testes e validações

### 12.1 Testes unitários

### 12.2 Testes de integração

|                                                   |       |
|---------------------------------------------------|-------|
| 12.3 Validação frontend                           | ..... |
| 12.4 Validação Docker executada                   | ..... |
| <b>13. Guia completo de execução</b>              | ..... |
| 13.1 Windows - PowerShell                         | ..... |
| 13.2 URLs principais                              | ..... |
| 13.3 Parar sem perder dados                       | ..... |
| <b>14. Roteiro de demonstração ao professor</b>   | ..... |
| 14.1 Comandos para demonstrar o banco             | ..... |
| <b>15. Diagnóstico e solução de problemas</b>     | ..... |
| 15.1 Logs                                         | ..... |
| <b>16. Decisões arquiteturais e limitações</b>    | ..... |
| 16.1 Decisões                                     | ..... |
| 16.2 Limitações assumidas                         | ..... |
| <b>17. Entrega e reprodução pelo avaliador</b>    | ..... |
| <b>Apêndice A - Exemplos completos de API</b>     | ..... |
| A.1 Criar pedido com PowerShell                   | ..... |
| A.2 Consultar e listar                            | ..... |
| A.3 Atualizar status                              | ..... |
| <b>Apêndice B - Comandos Docker de referência</b> | ..... |
| <b>Apêndice C - Glossário</b>                     | ..... |
| <b>Conclusão</b>                                  | ..... |

## 1. Contexto, objetivo e escopo

O trabalho implementa a primeira versão de uma aplicação de Pedidos. O domínio é simples por decisão pedagógica: a complexidade relevante está na separação de responsabilidades, na comunicação entre componentes, na persistência e na capacidade de reproduzir o ambiente com containers.

A arquitetura exigida é Cliente -> API de Pedidos -> PostgreSQL. O cliente conversa com a aplicação por HTTP/JSON; a aplicação conversa com o banco pelo protocolo PostgreSQL. API, Service e Repository são camadas lógicas do mesmo processo/container.

- Criar pedidos com cliente, produto, quantidade e valor unitário.
- Calcular valor\_total no backend, sem confiar em valor enviado pelo cliente.
- Definir automaticamente o status inicial CRIADO.
- Consultar um pedido por ID.
- Listar todos os pedidos.
- Atualizar somente o status de um pedido.
- Disponibilizar um endpoint de saúde.
- Persistir os dados em PostgreSQL de forma que sobrevivam ao reinício da aplicação.
- Executar aplicação e banco separadamente com Docker Compose.

**Escopo propositalmente excluído:** Autenticação, autorização, pagamento, estoque, múltiplos itens persistidos por pedido, CRUD de produtos, mensageria, paginação e microserviços adicionais não fazem parte desta entrega.

## 2. Atendimento aos requisitos da Entrega 1

| Requisito                                | Status   | Como foi implementado                                                         |
|------------------------------------------|----------|-------------------------------------------------------------------------------|
| Cliente -> API -> PostgreSQL             | Atendido | SPA/cliente acessa pedidos em 8000; aplicação acessa postgres na rede Docker. |
| API/Service/Repository no mesmo processo | Atendido | Camadas em app/api, app/services e app/repositories, empacotadas juntas.      |
| POST /pedidos                            | Atendido | Valida entrada, calcula total, define CRIADO, persiste e retorna 201.         |
| GET /pedidos/{id}                        | Atendido | Retorna 200 ou 404.                                                           |
| GET /pedidos                             | Atendido | Lista pedidos sem paginação.                                                  |
| PATCH /pedidos/{id}/status               | Atendido | Atualiza apenas status.                                                       |
| GET /health                              | Atendido | Resposta mínima {"status":"ok"}.                                              |
| PostgreSQL separado                      | Atendido | Serviço postgres:16 independente.                                             |
| Persistência após restart da API         | Atendido | Dados ficam no PostgreSQL/volume postgres_data.                               |

| Requisito                    | Status   | Como foi implementado                                              |
|------------------------------|----------|--------------------------------------------------------------------|
| Configuração por ambiente    | Atendido | DATABASE_URL obrigatória no backend; Compose fornece valor padrão. |
| Docker Compose               | Atendido | Serviços pedidos e postgres.                                       |
| Cliente acessa somente a API | Atendido | Porta 8000 publicada; 5432 não é publicada no host.                |

### 3. Arquitetura do sistema

A solução combina separação física entre aplicação e banco com separação lógica interna entre API, regras de negócio e persistência. Isso evita confundir "camada" com "container" e mantém a aplicação simples para a primeira entrega.

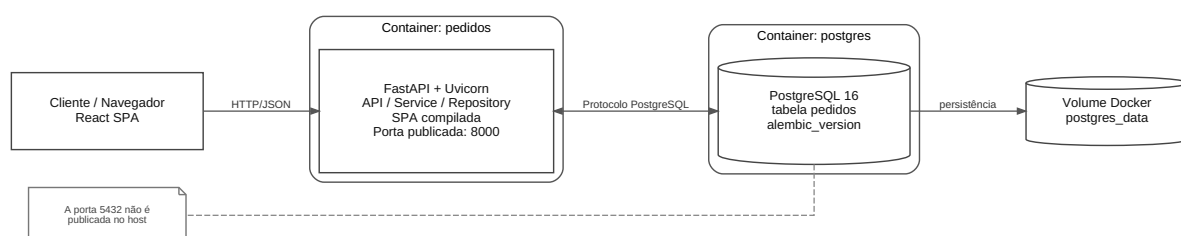


Figura 1 - Arquitetura física da solução.

#### 3.1 Arquitetura física

- Container pedidos: FastAPI, Uvicorn, camadas API/Service/Repository e build estático da SPA React.
- Container postgres: PostgreSQL 16.
- Rede Docker interna: usada para a conexão entre pedidos e postgres.
- Volume postgres\_data: armazena os dados persistentes do PostgreSQL.
- Porta publicada ao host: 8000, pertencente ao serviço pedidos.
- Porta 5432: disponível apenas internamente no container/rede; não publicada no host.

#### 3.2 Arquitetura lógica interna



Figura 2 - Camadas lógicas da aplicação de Pedidos.

| Camada       | Responsabilidade                                                                      | Não deve fazer                                    |
|--------------|---------------------------------------------------------------------------------------|---------------------------------------------------|
| API / Router | Receber HTTP, validar schemas, injetar dependências, produzir respostas e erros HTTP. | Concentrar regra de negócio ou SQL.               |
| Service      | Aplicar regras, calcular valor_total, definir CRIADO e coordenar operações.           | Conhecer detalhes HTTP ou implementar SQL direto. |

| Camada     | Responsabilidade                                             | Não deve fazer                      |
|------------|--------------------------------------------------------------|-------------------------------------|
| Repository | Persistir e consultar dados com SQLAlchemy; commit/rollback. | Decidir regra de negócio.           |
| PostgreSQL | Armazenar dados e garantir integridade relacional.           | Conhecer detalhes da interface web. |

### 3.3 Fluxo de criação de um pedido

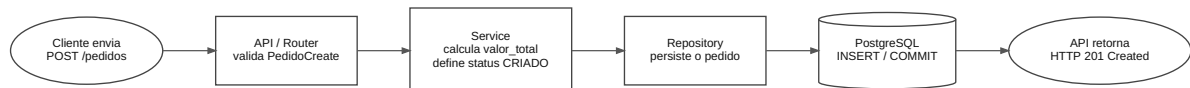


Figura 3 - Fluxo simplificado do POST /pedidos.

1. O cliente envia cliente, produto, quantidade e valor\_unitario.
2. O FastAPI/Pydantic valida o contrato de entrada.
3. O Service normaliza textos, calcula valor\_total e define status CRIADO.
4. O Repository persiste a entidade com SQLAlchemy.
5. O PostgreSQL confirma a transação.
6. A API retorna HTTP 201 com o pedido completo.

## 4. Organização do repositório

```
.
├── app/
│   ├── api/ # rotas HTTP
│   ├── models/ # modelos SQLAlchemy
│   ├── repositories/ # acesso a dados
│   ├── schemas/ # contratos Pydantic
│   ├── services/ # regras de negócio
│   ├── config.py # variáveis de ambiente
│   ├── database.py # engine, Session e Base
│   └── main.py # FastAPI + SPA fallback
├── alembic/
│   └── versions/
│       └── 0001_create_pedidos_table.py
├── docker/
│   └── entrypoint.sh
├── frontend/
├── tests/
│   ├── unit/
│   └── integration/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .env.example
├── alembic.ini
└── README.md
```

| Arquivo / pasta                       | Papel                                                                                 |
|---------------------------------------|---------------------------------------------------------------------------------------|
| app/main.py                           | Monta FastAPI, middleware CORS, rotas técnicas, arquivos estáticos e fallback da SPA. |
| app/api/                              | Endpoints /health e /pedidos.                                                         |
| app/services/pedido_service.py        | Regras de negócio do Pedido.                                                          |
| app/repositories/pedido_repository.py | Operações de banco e controle de transação.                                           |
| app/models/pedido.py                  | Mapeamento ORM da tabela pedidos.                                                     |
| app/schemas/pedido.py                 | Schemas de entrada/saída e validações.                                                |
| app/database.py                       | Engine SQLAlchemy e sessão por requisição.                                            |
| alembic/versions/                     | Histórico de migrations do banco.                                                     |
| docker/entrypoint.sh                  | Executa migration e inicia Uvicorn.                                                   |
| frontend/                             | SPA React/TypeScript.                                                                 |
| tests/                                | Testes unitários e de integração.                                                     |

## 5. Backend FastAPI

O backend utiliza Python 3.12, FastAPI, Uvicorn, Pydantic v2, SQLAlchemy 2.x, psycopg 3 e Alembic. A implementação é síncrona, adequada ao escopo da entrega e mais simples de compreender e depurar.

### 5.1 Inicialização da aplicação

- O objeto FastAPI define título, versão e OpenAPI.
- Swagger fica disponível em /docs e o schema em /openapi.json.
- ReDoc e o redirect OAuth2 do Swagger são desabilitados, reduzindo superfícies não utilizadas.
- Rotas de API são registradas antes do catch-all do frontend.
- Assets da SPA são servidos pela FastAPI quando o build está presente.
- Um handler genérico de exceção retorna HTTP 500 sem expor stack trace, SQL ou detalhes internos ao cliente.

### 5.2 Banco e sessão por requisição

A engine SQLAlchemy é criada uma vez por processo. SessionLocal fornece uma sessão por requisição FastAPI, fechada automaticamente ao final. pool\_pre\_ping=True ajuda a descartar conexões inválidas do pool.

**Configuração:** O código Python não contém host/usuário/senha de banco fixos. A conexão é recebida por DATABASE\_URL.

## 6. Modelo de domínio e regras de negócio

| Campo          | Tipo lógico | Origem / regra                                                              |
|----------------|-------------|-----------------------------------------------------------------------------|
| id             | inteiro     | Gerado pelo banco; chave primária.                                          |
| cliente        | texto       | Entrada obrigatória; normalizada pela aplicação.                            |
| produto        | texto       | Entrada obrigatória; normalizada pela aplicação.                            |
| quantidade     | inteiro     | Entrada; deve ser $\geq 1$ .                                                |
| valor_unitario | decimal     | Entrada; valor positivo, monetário.                                         |
| valor_total    | decimal     | Calculado no Service: quantidade x valor_unitario.                          |
| status         | texto       | Definido inicialmente como CRIADO; permite CRIADO, CONFIRMADO ou CANCELADO. |
| data_criacao   | timestamp   | Definido na criação/persistência.                                           |

### 6.1 Regras essenciais

- valor\_total nunca é aceito como dado de entrada do cliente.
- status inicial nunca é escolhido pelo cliente ao criar o pedido: sempre começa como CRIADO.
- id e data\_criacao são gerados pela aplicação/banco.

- O PATCH de status altera apenas o campo status.
- Valores monetários são manipulados com Decimal e armazenados como NUMERIC no PostgreSQL, evitando usar float como representação interna da regra de negócio.
- Campos extras nos schemas de entrada são rejeitados.

Exemplo:

quantidade = 2

valor\_unitario = 32.90

valor\_total = 65.80

status inicial = CRIADO

## 7. API REST e contratos HTTP

| Método | Rota                 | Finalidade               | Sucesso | Erros principais |
|--------|----------------------|--------------------------|---------|------------------|
| GET    | /health              | Health check             | 200     | -                |
| POST   | /pedidos             | Criar pedido             | 201     | 422              |
| GET    | /pedidos             | Listar pedidos           | 200     | -                |
| GET    | /pedidos/{id}        | Consultar pedido         | 200     | 404 / 422        |
| PATCH  | /pedidos/{id}/status | Atualizar somente status | 200     | 404 / 422        |

### 7.1 GET /health

GET /health

200 OK

```
{"status": "ok"}
```

É um endpoint técnico, não uma funcionalidade de negócio. Também é usado pelo healthcheck do container pedidos.

### 7.2 POST /pedidos

Entrada aceita:

```
{
  "cliente": "Ana Souza",
  "produto": "Combo de Hambúrguer",
  "quantidade": 2,
  "valor_unitario": 32.90
}
```

Resposta representativa:

```
{
  "id": 1,
  "cliente": "Ana Souza",
  "produto": "Combo de Hambúrguer",
  "quantidade": 2,
  "valor_unitario": 32.90,
  "valor_total": 65.80,
  "status": "CRIADO",
  "data_criacao": "<timestamp>"
}
```

### 7.3 GET /pedidos/{id}

Consulta sem modificar o estado. IDs inexistentes retornam 404. O projeto também faz validação estrita de IDs para evitar aceitar formatos ambíguos como 1e2 ou 1.5.

### 7.4 GET /pedidos

Retorna a coleção de pedidos existentes. Paginação não é exigida na primeira entrega e foi mantida fora do escopo.

### 7.5 PATCH /pedidos/{id}/status

```
{
  "status": "CONFIRMADO"
}
```

O contrato recebe somente o novo status e preserva cliente, produto, quantidade, valores e data de criação.

### 7.6 Documentação OpenAPI

O FastAPI gera automaticamente a especificação OpenAPI. O professor pode testar todos os endpoints em <http://localhost:8000/docs> sem instalar ferramentas adicionais.

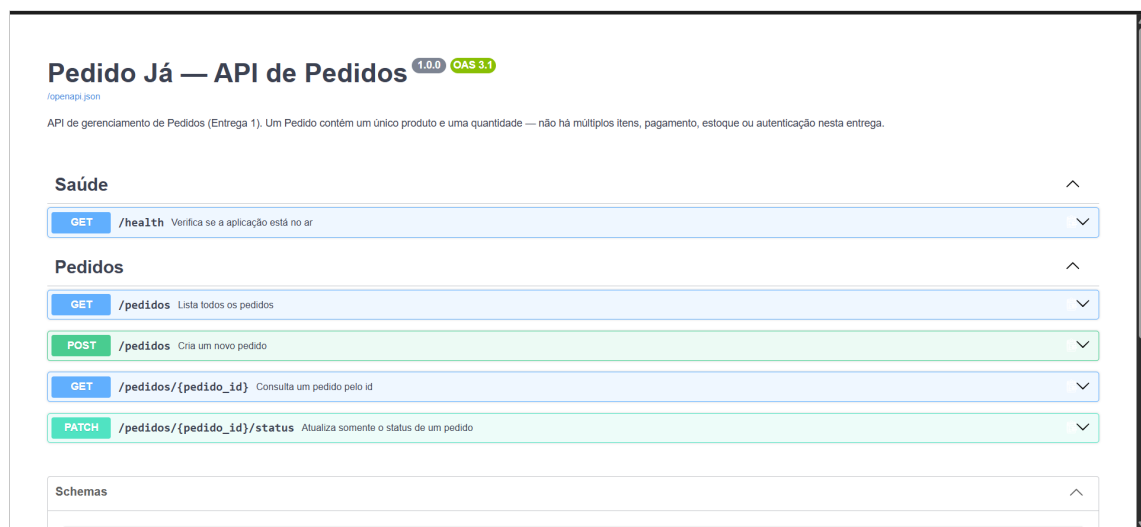


Figura - Swagger UI expondo apenas o health check e as operações previstas para Pedidos.

## 8. Persistência PostgreSQL e Alembic

O PostgreSQL é executado como componente independente, em container próprio. A aplicação não armazena pedidos em memória como fonte de verdade: todo pedido válido é persistido no banco.

### 8.1 Tabelas

| Tabela          | Finalidade                                                                 |
|-----------------|----------------------------------------------------------------------------|
| pedidos         | Tabela de negócio que armazena os pedidos.                                 |
| alembic_version | Tabela técnica utilizada pelo Alembic para registrar a migration aplicada. |

### 8.2 Estrutura lógica da tabela pedidos

| Coluna         | Tipo PostgreSQL | Observação                              |
|----------------|-----------------|-----------------------------------------|
| id             | INTEGER         | Primary key / autoincremento.           |
| cliente        | VARCHAR(120)    | Obrigatório.                            |
| produto        | VARCHAR(120)    | Obrigatório.                            |
| quantidade     | INTEGER         | CHECK para quantidade válida.           |
| valor_unitario | NUMERIC(12,2)   | Valor monetário.                        |
| valor_total    | NUMERIC(12,2)   | Valor monetário calculado.              |
| status         | VARCHAR(20)     | CHECK para CRIADO/CONFIRMADO/CANCELADO. |
| data_criacao   | TIMESTAMPTZ     | Momento de criação.                     |

### 8.3 Migrations

O schema é versionado por Alembic. A migration inicial 0001\_create\_pedidos\_table.py cria a tabela pedidos e suas restrições. Em produção não se depende de Base.metadata.create\_all() para montar o schema.

```
alembic upgrade head
```

Esse comando é executado automaticamente pelo endpoint do container pedidos antes do Uvicorn.

### 8.4 Persistência após reinício da aplicação

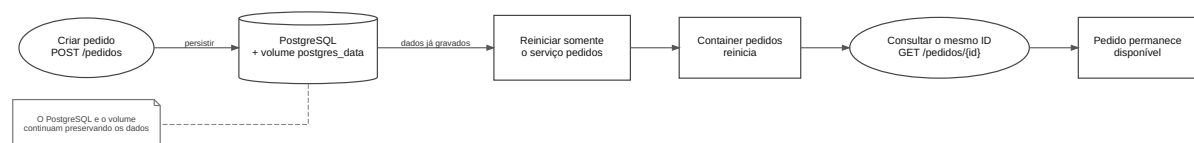


Figura 4 - Experimento de persistência exigido pela entrega.

**Por que o pedido permanece?:** Porque o restart afeta somente o processo/container da aplicação. O PostgreSQL continua sendo a fonte de verdade e seus arquivos de dados estão no volume postgres\_data.

## 9. Frontend React

A interface web é uma SPA em React + TypeScript + Vite. Ela oferece um fluxo público de pedido e uma área administrativa. No ambiente final, o build da SPA é copiado para a imagem Python e servido pela FastAPI.

### 9.1 Rotas públicas

| Rota                | Função                              |
|---------------------|-------------------------------------|
| /                   | Catálogo / página inicial.          |
| /produto/:produtoId | Detalhes do produto e quantidade.   |
| /carrinho           | Resumo do pedido em preparação.     |
| /revisar            | Revisão e identificação do cliente. |
| /acompanhar         | Busca/acompanhamento de pedido.     |
| /pedido/:id         | Visualização pública do pedido.     |

### 9.2 Rotas administrativas e técnicas

| Rota               | Função                             |
|--------------------|------------------------------------|
| /admin             | Visão geral administrativa.        |
| /admin/pedidos     | Lista de pedidos.                  |
| /admin/pedidos/:id | Detalhe e atualização de status.   |
| /admin/api         | Documentação visual dos endpoints. |
| /admin/arquitetura | Visão da arquitetura do sistema.   |

**Importante:** A área /admin é uma separação de UX, não autenticação. Login, usuários e autorização estão deliberadamente fora do escopo.

### 9.3 Integração com a API

- Produção usa same-origin: frontend e API respondem em localhost:8000.
- VITE\_API\_BASE\_URL é vazio no build de produção; não há URL localhost hard-coded para outro servidor.
- TanStack Query organiza consultas/mutações e estados de carregamento.
- O frontend usa parser estrito de IDs positivos para rotas de pedido.
- O catálogo de produtos é estático no frontend, pois CRUD de produtos não pertence à Entrega 1.

### 9.4 Evidências visuais da interface

As telas abaixo registram a integração do frontend com a API no ambiente final, incluindo o catálogo público, a área administrativa e o fluxo responsivo de criação do pedido.

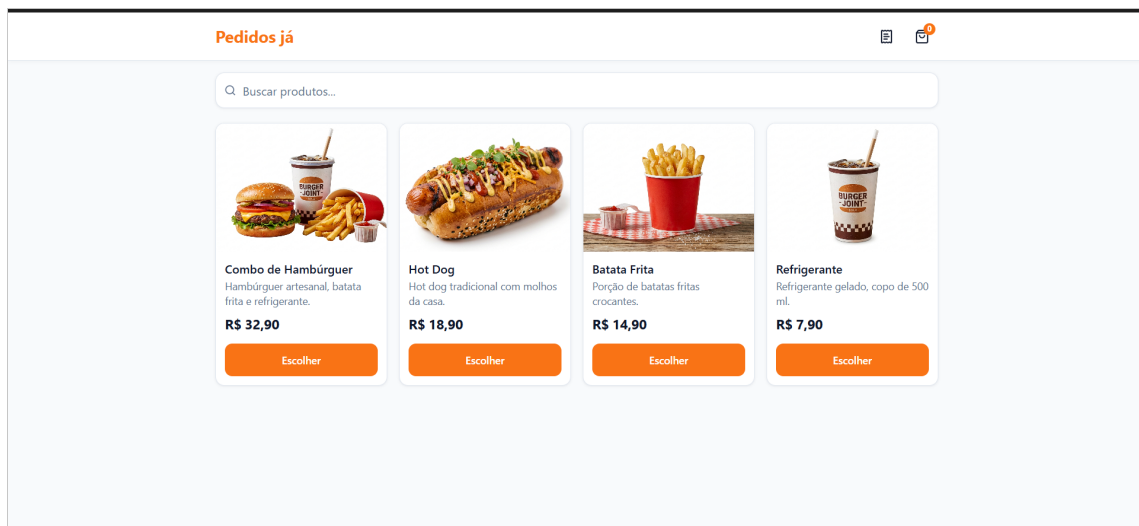


Figura - Catálogo público servido pela aplicação em `localhost:8000`.

**Pedido Já**  
Área Administrativa

**GESTÃO**

- Visão Geral
- Pedidos

**PROJETO**

- API & Documentação
- Arquitetura

Desenvolvimento de Sistemas Distribuídos

**Visão Geral**  
Panorama dos pedidos registrados na aplicação.

[Ver todos os pedidos →](#)

Pedidos  
**1**

Criados  
**1**

Confirmados  
**0**

Cancelados  
**0**

**Pedidos recentes**

| ID | CLIENTE                  | PRODUTO             | QTD. | VALOR UNITÁRIO | VALOR TOTAL      | STATUS              | DATA DE CRIAÇÃO     | AÇÕES                        |
|----|--------------------------|---------------------|------|----------------|------------------|---------------------|---------------------|------------------------------|
| #1 | Erick De Britto Carvalho | Combo de Hambúrguer | 1    | R\$ 32,90      | <b>R\$ 32,90</b> | <span>CRIADO</span> | 07/09/2026 às 23:50 | <a href="#">Ver detalhes</a> |

Projeto acadêmico — Desenvolvimento de Sistemas Distribuídos

API Arquitetura

Figura - Visão administrativa com indicadores e pedido persistido.

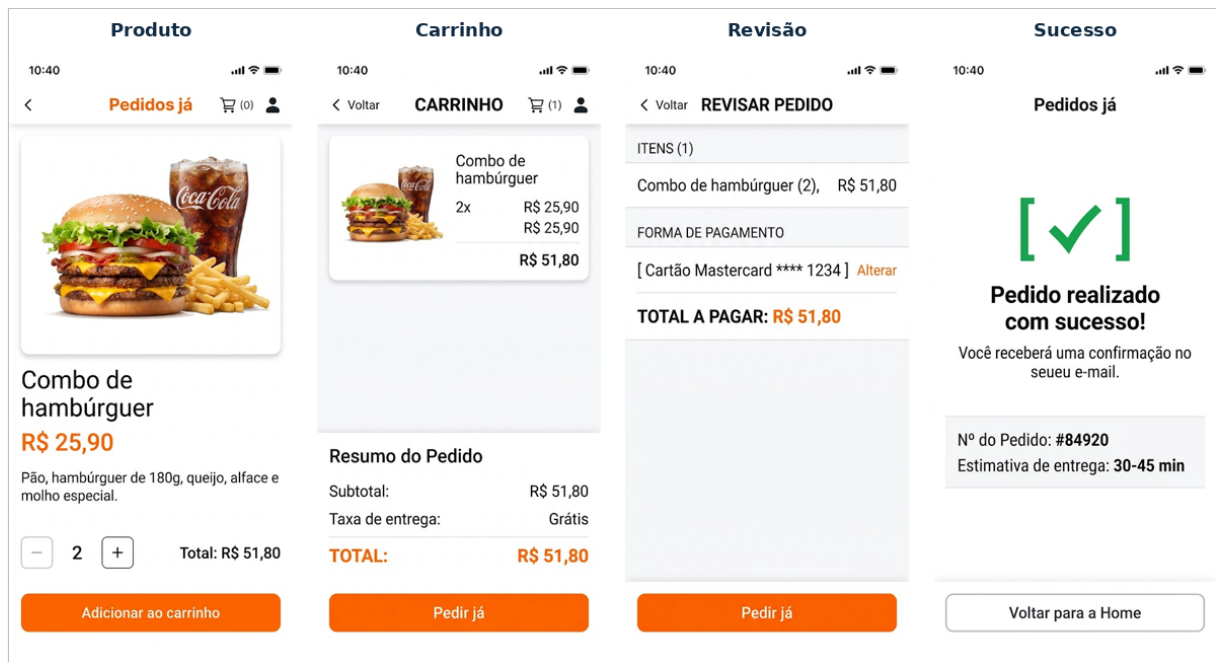


Figura - Evidência do fluxo responsivo: produto, carrinho, revisão e confirmação.

## 10. Docker e processo de inicialização

O Docker é utilizado para tornar o ambiente reproduzível e separar aplicação e banco. O Dockerfile é multi-stage e o docker-compose.yml descreve os dois componentes do sistema.

### 10.1 Dockerfile multi-stage

| Stage          | Imagem           | Função                                                                     |
|----------------|------------------|----------------------------------------------------------------------------|
| frontend-build | node:22-slim     | Executa npm ci e npm run build; gera frontend/dist.                        |
| backend        | python:3.12-slim | Instala dependências Python, copia app/ Alembic e recebe o dist compilado. |

Node não fica presente na imagem final de runtime. O container final executa como usuário não-root app.

### 10.2 Docker Compose

| Serviço  | Imagem/build     | Porta host   | Persistência                 | Dependência      |
|----------|------------------|--------------|------------------------------|------------------|
| postgres | postgres:16      | nenhuma      | postgres_data                | -                |
| pedidos  | Dockerfile local | 8000 -> 8000 | stateless; dados no postgres | postgres healthy |

### 10.3 Inicialização automatizada



Figura 5 - Fluxo de inicialização do ambiente Docker.

O entrypoint ainda possui retry para migrations como segurança adicional. Assim, a inicialização não depende de um simples sleep fixo.

## 11. Configuração por variáveis de ambiente

| Variável          | Valor padrão no Compose                                    | Uso                                   |
|-------------------|------------------------------------------------------------|---------------------------------------|
| DATABASE_URL      | postgresql+psycopg://pedidos:pedidos@postgres:5432/pedidos | Conexão do backend com PostgreSQL.    |
| CORS_ORIGINS      | http://localhost:5173                                      | Desenvolvimento do frontend separado. |
| POSTGRES_DB       | pedidos                                                    | Nome do banco.                        |
| POSTGRES_USER     | pedidos                                                    | Usuário do banco.                     |
| POSTGRES_PASSWORD | pedidos                                                    | Senha de laboratório do container.    |

Fora do Compose, DATABASE\_URL deve ser fornecida ao backend. Isso mantém host, porta e credenciais fora da lógica de aplicação e permite alterar topologia sem modificar código.

**.env.example:** Existe como referência de configuração. Na forma oficial de execução, não é necessário copiar manualmente .env.example para .env, pois o Compose já fornece os defaults.

## 12. Testes e validações

### 12.1 Testes unitários

Os testes unitários exercitam o PedidoService com repository fake, sem banco real. O foco é validar regra de negócio e colaboração entre camadas.

- cálculo de valor\_total com Decimal;
- status inicial CRIADO;
- normalização de cliente e produto;
- delegação correta ao Repository;
- atualização de status preservando demais dados.

### 12.2 Testes de integração

Os testes de integração usam FastAPI TestClient e PostgreSQL real de teste. TEST\_DATABASE\_URL é obrigatória e o nome do banco precisa terminar em \_test. A configuração de testes substitui DATABASE\_URL para reduzir o risco de executar limpeza contra o banco de desenvolvimento.

- POST /pedidos e persistência;
- GET por ID e listagem;
- PATCH de status;
- validações 422 e 404;
- rotas técnicas e proteção do fallback da SPA;
- serviço de assets/SPA usando fixture temporária, sem depender de um build frontend pré-existente em clone limpo.

### 12.3 Validação frontend

- Testes do parser de ID positivo/seguro.
- TypeScript typecheck.
- Lint.
- Build Vite de produção.

### 12.4 Validação Docker executada

O ambiente foi efetivamente construído e executado no Docker Desktop para Windows, e não apenas revisado estaticamente.

| Validação                    | Resultado observado                                                    |
|------------------------------|------------------------------------------------------------------------|
| docker compose config        | PASS - serviços pedidos e postgres, porta 8000 e volume postgres_data. |
| docker compose up -d --build | PASS - build multi-stage concluído e containers iniciados.             |
| docker compose ps            | PASS - pedidos e postgres healthy.                                     |
| GET /health                  | PASS - {"status":"ok"}.                                                |
| Frontend / Swagger / Admin   | PASS - páginas carregadas.                                             |
| Criação real de pedido       | PASS - pedido persistido no PostgreSQL.                                |
| Cálculo 2 x 32,90            | PASS - valor_total 65,80.                                              |
| PATCH para CONFIRMADO        | PASS - status atualizado sem alterar os demais campos.                 |
| restart somente pedidos      | PASS - pedido permaneceu disponível.                                   |
| stop/start somente pedidos   | PASS - postgres permaneceu ativo e dados preservados.                  |
| Porta 5432                   | PASS - não publicada no host.                                          |

## 13. Guia completo de execução

**Pré-requisito:** Docker Desktop/Engine e Docker Compose. Não é necessário instalar Python, Node ou PostgreSQL localmente para a execução oficial.

### 13.1 Windows - PowerShell

Abra o Docker Desktop, aguarde o Engine ficar ativo e abra o PowerShell na raiz do repositório, isto é, no diretório em que existe docker-compose.yml.

```
# confirmar que o Compose está na pasta atual
Test-Path .\docker-compose.yml
```

```
# iniciar tudo
docker compose up -d --build
```

```
# verificar containers
docker compose ps
```

```
# testar saúde
curl.exe http://localhost:8000/health
```

A resposta esperada do health check é:

```
{"status":"ok"}
```

## 13.2 URLs principais

| URL                                | Conteúdo                  |
|------------------------------------|---------------------------|
| http://localhost:8000/             | Interface web do cliente. |
| http://localhost:8000/admin        | Área administrativa.      |
| http://localhost:8000/docs         | Swagger UI.               |
| http://localhost:8000/openapi.json | Schema OpenAPI.           |
| http://localhost:8000/health       | Health check.             |
| http://localhost:8000/pedidos      | Coleção da API.           |

## 13.3 Parar sem perder dados

```
docker compose down
```

Esse comando remove os containers/rede do Compose, mas preserva o volume nomeado do banco.

**Atenção:** Evite `docker compose down -v`. O parâmetro `-v` remove o volume e apaga os pedidos persistidos.

## 14. Roteiro de demonstração ao professor

1. Abrir Docker Desktop e confirmar Engine running.
2. Abrir PowerShell na raiz do projeto.
3. Executar `docker compose up -d --build`.
4. Executar `docker compose ps` e mostrar somente pedidos e postgres, ambos healthy.
5. Executar `curl.exe http://localhost:8000/health`.
6. Abrir `/`, `/docs` e `/admin` no navegador.
7. Criar um pedido no frontend com quantidade 2.
8. Mostrar que 2 x R\$ 32,90 resulta em R\$ 65,80 e status inicial CRIADO.
9. No admin, alterar o status para CONFIRMADO.

10. Mostrar o registro diretamente no PostgreSQL.
11. Executar docker compose restart pedidos.
12. Consultar novamente o mesmo pedido e mostrar que permaneceu CONFIRMADO.
13. Opcional: docker compose stop pedidos; docker compose ps para demonstrar postgres ativo; depois docker compose start pedidos.

### 14.1 Comandos para demonstrar o banco

```
docker compose exec postgres psql -U pedidos -d pedidos -c "\dt"
```

```
docker compose exec postgres psql -U pedidos -d pedidos -c "\d+ pedidos"
```

```
docker compose exec postgres psql -U pedidos -d pedidos -c "SELECT id, cliente, produto, quantidade, valor_unitario, valor_total, status, data_criacao FROM pedidos ORDER BY id;"
```

```
docker compose exec postgres psql -U pedidos -d pedidos -c "SELECT * FROM alembic_version;"
```

## 15. Diagnóstico e solução de problemas

| Sintoma                          | Verificação                             | Ação                                                                                |
|----------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------|
| "no configuration file provided" | Test-Path .\docker-compose.yml          | Entrar na raiz do projeto antes de usar docker compose.                             |
| localhost:8000 não responde      | docker compose ps                       | Confirmar que pedidos está Up/healthy.                                              |
| pedidos reinicia ou falha        | docker compose logs pedidos --tail 100  | Ver migrations, DATABASE_URL e erro da aplicação.                                   |
| postgres não fica healthy        | docker compose logs postgres --tail 100 | Ver inicialização do banco/volume/credenciais.                                      |
| Porta 8000 ocupada               | netstat -ano   findstr :8000            | Parar o processo conflitante antes de subir o Compose.                              |
| Frontend retorna 404 de build    | logs do pedidos                         | Reconstruir com docker compose up -d --build.                                       |
| Acentos estranhos no PowerShell  | navegador exibe corretamente            | Usar corpo UTF-8 em Invoke-RestMethod; pode ser limitação visual do console antigo. |

### 15.1 Logs

```
docker compose logs pedidos --tail 100
```

```
docker compose logs postgres --tail 100
```

```
docker compose logs -f pedidos
```

```
# Ctrl+C encerra o acompanhamento em tempo real
```

## 16. Decisões arquiteturais e limitações

### 16.1 Decisões

| Decisão                | Justificativa                                                                   |
|------------------------|---------------------------------------------------------------------------------|
| Arquitetura em camadas | Separa HTTP, regra de negócio e persistência sem criar serviços desnecessários. |
| PostgreSQL separado    | Representa componente/processo independente e permite persistência real.        |
| Decimal + NUMERIC      | Evita problemas típicos de ponto flutuante em valores monetários.               |
| Alembic                | Versiona o schema e torna a preparação do banco reproduzível.                   |
| SPA same-origin        | Mantém dois containers apenas e simplifica integração frontend/API.             |
| Sem porta 5432 no host | Cliente acessa somente a API, conforme a arquitetura proposta.                  |
| Dockerfile multi-stage | Node é usado somente no build; runtime final é menor e mais simples.            |
| Healthchecks           | Permitem coordenação do startup e observação de disponibilidade.                |

### 16.2 Limitações assumidas

- Sem autenticação, autorização, login ou JWT.
- Sem usuários persistidos.
- Sem múltiplos itens persistidos no mesmo pedido.
- Sem pagamento e sem frete.
- Sem estoque e disponibilidade.
- Sem CRUD de produtos no backend; catálogo estático no frontend.
- Sem paginação/filtros server-side.
- Sem PUT/DELETE de pedidos.
- Sem Redis, Kafka, RabbitMQ ou outros componentes distribuídos nesta etapa.

## 17. Entrega e reprodução pelo avaliador

A solução foi preparada para ser reproduzida a partir do repositório sem configuração manual de banco. A tag prevista pela atividade para a Entrega 1 é APIPedidos-1-final.

```
git clone <URL_DO_REPOSITORIO>
cd <repositorio>
git checkout APIPedidos-1-final
docker compose up -d --build
```

Depois, o avaliador pode confirmar:

- GET `http://localhost:8000/health` retorna status ok.
- Swagger disponível em `/docs`.
- POST `/pedidos` persiste dados.
- GET `/pedidos/{id}` recupera o registro.
- GET `/pedidos` lista registros.
- PATCH `/pedidos/{id}/status` altera somente o status.
- docker compose restart pedidos não apaga pedidos.

**Condição de sucesso:** Se o clone/tag sobe com um único docker compose up -d --build e o pedido sobrevive ao restart de pedidos, os principais critérios operacionais da Entrega 1 estão comprovados.

## Apêndice A - Exemplos completos de API

### A.1 Criar pedido com PowerShell

```
$body = @{
cliente = "Teste Final Docker"
produto = "Combo de Hambúrguer"
quantidade = 2
valor_unitario = 32.90
} | ConvertTo-Json -Compress

$bodyUtf8 = [System.Text.Encoding]::UTF8.GetBytes($body)

$pedido = Invoke-RestMethod `
-Method Post `
-Uri "http://localhost:8000/pedidos" `
-ContentType "application/json; charset=utf-8" `
-Body $bodyUtf8

$pedido | Format-List
$Id = $pedido.id
```

### A.2 Consultar e listar

```
Invoke-RestMethod "http://localhost:8000/pedidos/$Id" | Format-List
Invoke-RestMethod "http://localhost:8000/pedidos" | Format-Table
```

### A.3 Atualizar status

```
$statusBody = [System.Text.Encoding]::UTF8.GetBytes('{ "status": "CONFIRMADO" }')

Invoke-RestMethod `
-Method Patch `
-Uri "http://localhost:8000/pedidos/$Id/status" `
-ContentType "application/json; charset=utf-8" `
-Body $statusBody
```

## Apêndice B - Comandos Docker de referência

| Objetivo         | Comando                                     |
|------------------|---------------------------------------------|
| Validar Compose  | <code>docker compose config</code>          |
| Subir/rebuild    | <code>docker compose up -d --build</code>   |
| Ver estado       | <code>docker compose ps</code>              |
| Reiniciar só API | <code>docker compose restart pedidos</code> |
| Parar só API     | <code>docker compose stop pedidos</code>    |

| Objetivo                       | Comando                                              |
|--------------------------------|------------------------------------------------------|
| Iniciar só API                 | <code>docker compose start pedidos</code>            |
| Logs API                       | <code>docker compose logs pedidos --tail 100</code>  |
| Logs banco                     | <code>docker compose logs postgres --tail 100</code> |
| Desligar mantendo volume       | <code>docker compose down</code>                     |
| APAGAR volume - evitar em demo | <code>docker compose down -v</code>                  |

## Apêndice C - Glossário

| Termo         | Significado no projeto                                                         |
|---------------|--------------------------------------------------------------------------------|
| API           | Interface HTTP exposta pelo FastAPI.                                           |
| Service       | Camada que concentra regras de negócio.                                        |
| Repository    | Camada que abstrai persistência.                                               |
| ORM           | Mapeamento entre objetos Python e tabelas SQL via SQLAlchemy.                  |
| Migration     | Alteração versionada do schema do banco, gerenciada pelo Alembic.              |
| Healthcheck   | Verificação automática de disponibilidade de um serviço.                       |
| SPA           | Single Page Application; frontend React com roteamento client-side.            |
| Same-origin   | Frontend e API publicados na mesma origem/porta.                               |
| Volume Docker | Armazenamento persistente independente do ciclo de vida do container.          |
| Stateless     | Aplicação não depende de memória local para preservar pedidos entre reinícios. |

## Conclusão

O Pedido Já implementa a primeira entrega como uma aplicação distribuída simples, reproduzível e alinhada ao objetivo arquitetural da disciplina. O backend mantém responsabilidades separadas em API, Service e Repository; o PostgreSQL é um componente independente e persistente; o ambiente é automatizado por Docker Compose; e o frontend integra o fluxo funcional sem introduzir serviços adicionais fora do escopo.

A solução também demonstra na prática o ponto central do experimento da disciplina: reiniciar a aplicação não remove os pedidos, pois os dados pertencem ao PostgreSQL e ao seu volume persistente. Essa separação prepara a base para evoluções futuras da arquitetura sem antecipar complexidade não solicitada na Entrega 1.

**Execução oficial:** `docker compose up -d --build`